FORMATTING INSTRUCTIONS FOR ICLR 2025 CONFERENCE SUBMISSIONS

Anonymous authors
Paper under double-blind review

ABSTRACT

Text-to-speech (TTS) systems can read Vietnamese aloud, but they cannot yet reliably make non-verbal sounds, such as laughing, sighing or coughing, at the point where the text asks for them. These sounds make synthetic speech feel human in voice assistants, customer-facing AI agents, audiobooks and videos. Two things stand in the way: existing systems ignore or garble such tags in Vietnamese, and no Vietnamese training data with labelled sounds exists. We build that data automatically by adapting the SynParaSpeech method to Vietnamese: recordings of real sounds are converted into each speaker's voice and inserted into natural pauses in real Vietnamese speech, with a matching tag in the transcript. The result is about 811 hours of labelled speech covering 17 sound types. Fine-tuning the open-source OmniVoice model on this data teaches it 15 new tags. The fine-tuned model makes the right sound at the right place for 50% of tags, against 11.5% for the released model, and misses only 35% of them, against 64%. Its word error rate on the tagged sentences falls from 8.1% to 6.7%; on plain text it rises a little, from 3.9% to 4.8%, with the same speaker similarity, and it runs at a real-time factor of 0.06, fast enough for live conversation.

1 INTRODUCTION

When people talk, they communicate with more than words. A short laugh, a sigh, a breath before a long sentence or a hesitant "ừm…" tells the listener how the speaker feels and what they mean. These paralinguistic, or non-verbal, cues, which we call non-verbal vocalizations (NVVs), run through everyday conversation, and speech without them sounds flat and mechanical even when every word is pronounced correctly.

Text-to-speech (TTS) systems turn written text into spoken audio. Modern TTS models read text clearly in many voices, and some now let the user place a tag in the text, as in "Thật à [laughter] mình không ngờ luôn" ("Really? [laughter] I didn't expect that at all"), so that the voice laughs at exactly that point. This control is what makes synthetic speech usable where expression matters. A voice agent that answers customers can chuckle at a joke or sigh in sympathy at a complaint, instead of meeting both in the same even tone, and that small reaction is often what makes a caller feel heard rather than processed. An audiobook, a dubbed video or a podcast can let its narrator gasp, laugh or catch a breath where the script calls for it, which holds a listener's attention far better than a steady, unbroken read.

For Vietnamese, this control does not exist yet. What is needed is a TTS system that, given a Vietnamese sentence with a tag such as `[cough]`, makes that sound at that point in the sentence, in the chosen voice, as naturally as a person would. To our knowledge, no available system can do this. Systems that accept such tags are built mainly for other languages: Fish Audio's open model, OpenAudio S1 (Fish Audio, 2025), offers markers such as `(laughing)` and `(sighing)` in 13 languages, none of them Vietnamese. OmniVoice (Zhu et al., 2026) does speak Vietnamese, but its tags were learned from Chinese and English data, and in Vietnamese it often ignores them, reads them out as words or produces broken audio around them. In our tests on Vietnamese sentences, the released OmniVoice made no sound at all for 64% of the tags, and the right sound in the right place for only 11.5% (§3.5). The natural remedy, training a model on Vietnamese speech in which each sound is labelled at its position, is blocked by a lack of data: to our knowledge, no public Vietnamese dataset of this kind exists.

This work closes both gaps. We start from OmniVoice, an open-source TTS model that already speaks Vietnamese and runs many times faster than real time, which makes it practical to deploy in live conversation. Of the sounds we need, it supports only laughter and sigh, so we teach it 15 new tags: cough, throat clearing, sneeze, sniff, breathing, gasp, moan, grunt, burp, yawn, crying, pant, yell, screaming and snoring. Because no suitable data existed, we build it by adapting SynParaSpeech (Bai et al., 2025), a published method that creates such data automatically by inserting recorded sounds into real speech, to Vietnamese, and we extend it in five ways. We collect 17 sound types instead of 6, keeping only clean recordings from several public collections. An audio language model listens to each recording and places the tag among the speaker's real pauses, where SynParaSpeech uses a text-only model that reads the transcript. Each candidate recording of the sound is scored on how well its loudness, recording quality and rhythm match the speaker, where SynParaSpeech picks one at random. The inserted sound is set to a suitable level and given as much silence as the speaker normally pauses for, filled with the recording's own background noise. Finally, every example is scored by an automatic quality predictor and dropped if the edit damaged the audio, where SynParaSpeech relies on manual review. We then fine-tune OmniVoice on the resulting dataset, that is, train it further on our data.

Our contributions are as follows:

- A Vietnamese NVV data pipeline that adapts SynParaSpeech to Vietnamese and extends it with a larger sound inventory, audio-aware tag placement, speaker-matched clip selection, natural pauses and automatic quality filtering (§2.1).
- A Vietnamese NVV dataset of about 811 hours (224,064 utterances) covering 17 sound types, with 10,000 to 13,500 utterances per type and the position of each sound labelled in the transcript.
- A fine-tuned OmniVoice that makes the right sound at the right place for 50% of tags, against 11.5% for the released model, and lowers the word error rate on tagged sentences from 8.1% to 6.7%, at a small cost on plain text (WER 4.8% against 3.9%, with the same speaker similarity), while staying fast, with a real-time factor (the time taken to generate speech divided by the length of that speech) of 0.06, the same as the released model.

2 METHOD

Our system extends a pretrained zero-shot TTS model with control over non-verbal vocalizations (NVV) in Vietnamese. Since Vietnamese speech with NVV annotations is scarce, we synthesize training data by inserting voice-converted NVV clips into real Vietnamese recordings at natural pauses, with an inline tag at the matching position in the transcript (§2.1). We then fine-tune the TTS model on this data mixed with untagged Vietnamese speech (§2.2). At inference, an inline tag such as `[laughter]` in the input text produces the corresponding vocalization in the cloned voice. Figure 1 gives an overview.

2.1 TRAINING-DATA SYNTHESIS

Each training example is a real Vietnamese utterance with one NVV event spliced into one of its pauses and a tag inserted at the same point in its transcript. It is built from a pool of NVV clips, collected once, in six steps per utterance (Figure 1a). Appendix A gives the settings of each part.

- **NVV clips.** Built once, before synthesis. VocalSound (Gong et al., 2022) alone is limited: it covers six classes (laughter, sigh, cough, throat clearing, sneeze, sniff), and its crowd-sourced clips often carry long silences or stray speech from a loose cut. To add classes and keep only clean events, we pool clips from other NVV corpora and from events cut out of a tagged speech corpus. Where a source labels whole clips or whole captions rather than events, the events are first located in time. Every event is then trimmed and kept only if it is a clean, isolated instance of its class: a sound event detector confirms its class and no other, no speech overlaps it, its length is plausible for the class, and it is neither buried in noise nor clipped. The pool covers 17 classes (Table 5).
- **Slot finding.** Following SynParaSpeech (Bai et al., 2025), we first find the pauses where an NVV could go and only then let a model choose among them. Placing the tag first and

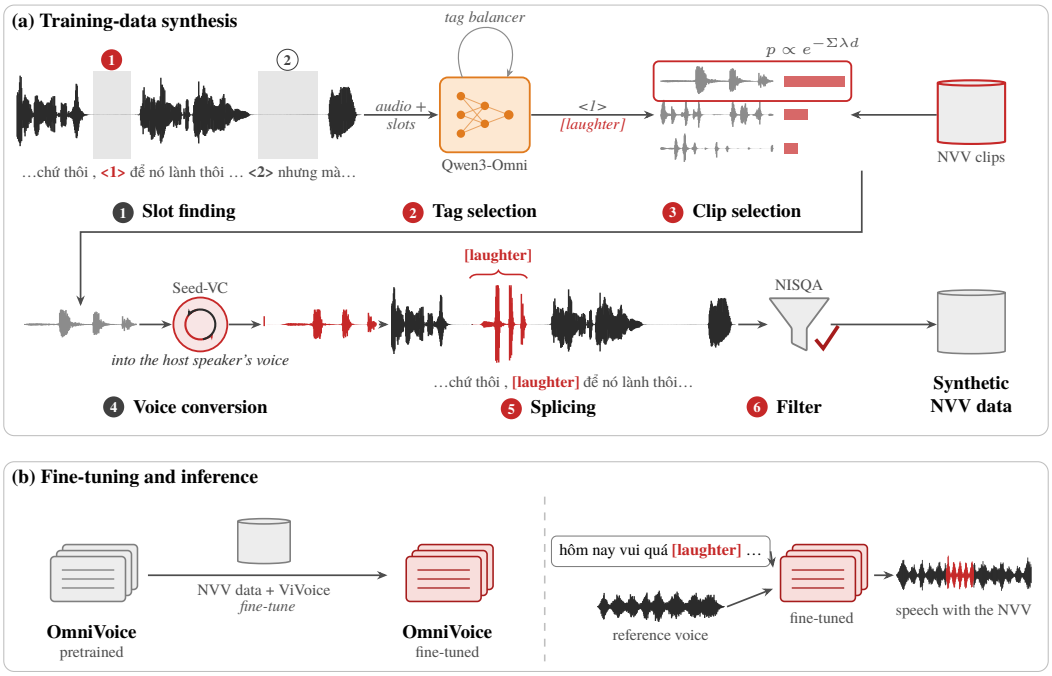


Figure 1: Overview. (a) Training-data synthesis on a real example: VAD pauses between aligned words become slots; an omni LLM picks one slot and one tag; an NVV clip of that class is sampled to match the speaker, converted to the speaker's voice with Seed-VC, spliced into the pause and quality-filtered. Red step numbers mark the steps we change from SynParaSpeech, and the red outline marks the extended NVV clip pool. (b) OmniVoice is fine-tuned on the synthetic data with untagged ViVoice; at inference an inline tag produces the NVV in the cloned voice (waveforms in (b) are illustrative).

aligning it afterwards fails on connected speech, where the boundary between two words is usually continuous voicing. A voice activity detector finds the pauses, a forced aligner finds the word boundaries, and a pause that falls between two words becomes a *slot*. The slots are marked with numbered markers in the transcript, e.g. "… chứ thôi , <1> để nó lành thôi … <2> nhưng mà …" in Figure 1a.

– **Tag selection.** An omni-modal LLM listens to the recording, reads the marked transcript, and picks the one slot where a non-verbal sound would most plausibly occur, together with the tag that fits what the speaker is doing there, or declines if no slot suits any sound. The LLM never writes transcript text: the tag is inserted in code at the chosen slot and the event is later spliced at the slot's midpoint, so text and audio refer to the same point by construction. Left alone, the LLM settles on a few neutral classes, so a tag balancer withholds any tag that already holds too large a share of the assignments until the others catch up.

– **Clip selection.** Within the chosen class, a clip is drawn at random, weighted towards clips that match the host speaker on the properties that voice conversion leaves unchanged. Timbre and pitch are rebuilt by the conversion, so we do not match on them. Instead we compare the clip with the speech just before the cut on four measures: *intensity*, *clarity* (the recording's signal-to-noise ratio), *tempo* (for laughter only) and *tilt* (whether the energy rises or decays). With each distance d_a normalised by a per-measure tolerance and weighted by λ_a, a clip is drawn with probability proportional to

$$w = \exp\Big(-\sum_a \lambda_a d_a\Big). \tag{1}$$

Sampling, rather than taking the closest clip, keeps every usable clip in play, so a soft-spoken speaker still sometimes gets a loud laugh.

– **Voice conversion.** The selected clip is converted to the host speaker's voice with Seed-VC, using the host utterance itself as the voice reference. Pitch conditioning is off, since its pitch tracker is built for speech and singing and fails on coughs, sniffs and throat clearing.

– **Splicing.** The converted event is scaled to the loudness of the speech before the cut, with a per-class offset (a sneeze is louder than a sniff), and spliced in at the slot's midpoint with short exponential fades, which keep the sharp onset of a laugh or cough that a cosine fade would soften. Each side of the event should then carry about as much silence as the speaker normally pauses for. The target pauses are

$$\begin{aligned} T_{\text{pre}} &= \bar{T}_p \cdot s_g \cdot s_{\text{pre}}^{(c)} \\ T_{\text{post}} &= \bar{T}_p \cdot s_g \cdot s_{\text{post}}^{(c)} \end{aligned} \tag{2}$$

where $\bar{T}_p$ is the speaker's median pause in the utterance, s_g is a global scale, and $s_{\text{pre}}^{(c)}$ and $s_{\text{post}}^{(c)}$ are scales for class c: events that end on emptied lungs, such as a laugh or a sigh, need a longer beat after them. Silence is only added up to the target, so the speaker's own pause is never shortened, and the added silence is filled with the recording's own background noise.

– **Filter.** Each finished example is scored with NISQA v2.0 (Mittag et al., 2021), a no-reference predictor of the mean opinion score (MOS) and of four quality dimensions, on both the source recording and the spliced one. It is kept only if neither its overall quality nor its continuity drops by more than a threshold. The thresholds apply to the drop rather than to absolute scores, because an absolute floor mostly measures how clean the source corpus was. A further check rejects cuts that land inside loud speech, which NISQA no longer hears once the fades and the added silence have smoothed the junction.

2.2 Fine-tuning and inference

We build on OmniVoice (Zhu et al., 2026), a massively multilingual zero-shot TTS model that maps text directly to multi-codebook acoustic tokens with a non-autoregressive diffusion language model and clones a voice from a reference recording. The released model cannot generate Vietnamese NVVs as it stands, for three reasons. First, its tag set is small: only two of its inline tags, `[laughter]` and `[sigh]`, are non-verbal sounds, the rest being lexical interjections such as `[question-ah]`, so 15 of our 17 classes have no tag at all. Second, it has never seen a tag in Vietnamese: Vietnamese speech, ViVoice among it, is in its training data but untagged, and its tagged data come from Chinese and English corpora (NVSpeech, NVS-38K and ParaSpeechCaps). Third, it was not trained for tag control: these corpora are a small fraction of its training data, and the OmniVoice paper does not evaluate tag control. In practice the released model often drops the tag in Vietnamese text, even for the two classes it supports (§3.5).

We therefore fine-tune OmniVoice on the synthetic data of §2.1, which supplies all 17 tags on Vietnamese speech, mixed with untagged Vietnamese speech to preserve ordinary synthesis. Nothing else changes: the training objective is OmniVoice's own, and at inference the tag is written inline in the input text, as with OmniVoice's own tags.

3 Experiments

3.1 Data

Training uses ViVoice (capleaf, 2024) and the synthetic NVV set of §2.1: 224,064 utterances (810.8 h), each with one event from the 17 classes of Table 5; Table 1 breaks it down by tag. Each training batch mixes synthetic and untagged utterances at a ratio of XX:XX. VocalSound clips are drawn from its test and validation splits only (5,446 clips, about 907 per class, with no speaker in common), so a vocal-sound classifier trained on VocalSound's training split can still be evaluated fairly on our output. The other NVV sources are used in full. All audio is resampled to XX kHz.

Table 1: Synthetic NVV training set per tag, after quality filtering. Each utterance carries one event. Hours are the total duration of the finished utterances.

Tag	Utterances	Hours	Tag	Utterances	Hours
`laughter`	13,450	55.20	`grunt`	13,442	44.11
`sigh`	13,455	52.46	`burp`	13,435	41.82
`cough`	13,440	49.20	`yawn`	13,433	50.59
`throatclearing`	13,441	48.69	`crying`	13,433	51.01
`sneeze`	13,435	49.55	`pant`	13,432	49.94
`sniff`	13,522	51.70	`yell`	13,400	45.33
`breathing`	13,452	47.69	`screaming`	12,212	49.42
`gasp`	13,449	45.10	`snoring`	10,190	31.69
`moan`	13,443	47.33			
Total				224,064	810.82

3.2 Training

We initialise from the public OmniVoice checkpoint (k2-fsa/OmniVoice) and fine-tune all parameters with AdamW (β_1=0.9, β_2=0.98), peak learning rate XX, XX warm-up steps and decay, for XX steps with a batch of XX s of audio per GPU, in bf16. Training takes XX h on NVIDIA B200. The training objective is unchanged from OmniVoice. We select the final checkpoint.

3.3 Baselines

We evaluate two tasks. In zero-shot TTS, the input is plain Vietnamese text without tags, spoken in a voice cloned from a reference recording; this task checks that fine-tuning for NVVs keeps ordinary Vietnamese synthesis intact. Its baselines are the released OmniVoice and four Vietnamese TTS systems, viXTTS, F5-TTS, VietVoice-TTS and VieNeu-TTS, all evaluated on the same test subset (Table 2). In NVV synthesis, the input is tagged Vietnamese text, and each tag must become the matching vocalization at its position. Its baselines are the released OmniVoice, given the same tagged input as our model, and Fish Audio's commercial TTS service (Table 3); the four Vietnamese systems have no NVV tags and are not compared on this task. The listening test on NVV synthesis (Table 4) covers three systems: Fish Audio's commercial TTS service, a multilingual system with its own expression markers, the released OmniVoice and our fine-tuned model.

3.4 Evaluation

For zero-shot TTS, every system, the four Vietnamese systems included, is evaluated on the same 238 utterances sampled from VietVoice, an internal Vietnamese speech dataset, so the rows of Table 2 compare directly.

For NVV synthesis, we use 420 real Vietnamese sentences, 30 for each of 14 classes; burp, grunt and yell are not tested. Each sentence is chosen so that the tag fits its content, for example a sentence in which someone bursts out laughing for `[laughter]`, and the tag sits where the sound would naturally occur (Appendix B.1). Each sentence is synthesised once with each of 7 reference voices, prompted with a reference recording and its transcript. None of the 7 voices appears in the training data, so every result is for a voice the model has never heard. Each system therefore produces 2,940 items (420 sentences $\times$ 7 voices). Every objective metric is computed over all items of its task; the listening test uses a subset of the NVV items.

Both OmniVoice models use its voice-cloning inference with the language set to Vietnamese, and the same item gets the same random seed in both. The released inference code tokenises an inline tag apart from the text around it, whereas training tokenises the whole string, so a model can miss or garble tags it has learned. We therefore tokenise the whole string at inference for both models, the released one included, so that they receive the same input.

We report five objective metrics, each on its own as the mean over all items of a system unless stated otherwise; no combined score is computed. NVPA, WER and SIM-o follow Track A of the VLSP 2026 NVV TTS task, and Appendix B.2 gives the full procedures and model versions.

NV placement accuracy (NVPA) measures whether the tagged vocalization is produced where the tag sits. A sound event detector (FlexSED) finds the vocalizations in the output, and a Vietnamese CTC forced aligner places the tag among the spoken words; a tag counts as correct when a vocalization of its class is found at its position. NVPA is the share of correct tags, from 0 to 1, where 1 means every tag became the right sound in the right place. The NV miss rate (NVMR) is the share of tags at which no vocalization of any class is found, because the model skipped the tag or read it out as words. NVPA also counts a wrong or misplaced vocalization as a failure, so NVMR isolates the tags that were ignored; it also runs from 0 to 1, and lower is better.

Word error rate (WER) measures intelligibility: a Vietnamese ASR model (Zipformer) transcribes the output, and the transcript is compared with the input text, tags removed. It is the share of words substituted, deleted or inserted, in percent; lower is better, 0% means every word was recognised, and it can exceed 100% when many words are inserted. Speaker similarity (SIM-o) measures how well the voice is cloned, as the cosine between the speaker embeddings (ECAPA-TDNN) of the output and of the reference recording used as the prompt. It ranges from -1 to 1, in practice from 0 to 1, and values near 1 mean the output sounds like the reference speaker. The real-time factor (RTF) measures speed: the time taken to synthesise an utterance divided by the duration of the audio produced, on one NVIDIA A100 GPU (DGX A100). Lower is better, and below 1 is faster than real time; an RTF of 0.5, for example, produces 10 s of speech in 5 s. We report the mean over items.

The listening test covers NVV synthesis only. Native Vietnamese listeners rate samples from three systems, Fish Audio, the released OmniVoice and our fine-tuned model, hearing one sample at a time and giving it three mean opinion scores (MOS) on a five-point scale (1 = bad, 5 = excellent; higher is better). MOS-Q rates quality: how clean the audio is, free of noise, distortion and glitches. MOS-N rates naturalness: how much the speech sounds like a real person talking, including whether the vocalization sounds real and sits naturally in the sentence. MOS-E rates expressiveness: how well the sample conveys the reaction the text calls for, that is, whether the vocalization is the one the tag asks for and carries the feeling of the sentence, rather than sounding flat or out of place. XX listeners rate XX samples per system, and we report each score as a mean with its 95% confidence interval (Table 4).

3.5 RESULTS

Table 2 compares our fine-tuned model with the released OmniVoice and other Vietnamese TTS systems on zero-shot synthesis of plain text, without tags, with all six systems evaluated on the same 238 utterances. Table 3 compares the released and fine-tuned OmniVoice on NVV synthesis, and Table 4 gives the listening-test scores on NVV synthesis. Given Vietnamese text, the released model produces no vocalization at 37% of the `[laughter]` and `[sigh]` tags, the two it supports, and at 69% of the other 12; overall it realises only 11.5% of tags correctly. Even its own two tags behave very differently: it produces a correct laugh for 79% of `[laughter]` tags but a correct sigh for only 1% of `[sigh]` tags.

Fine-tuning on our data changes this. The share of missed tags falls from 64% to 35%: from 37% to 10% on `[laughter]` and `[sigh]`, and from 69% to 39% on the 12 tags the released model never learned. The share of tags realised correctly rises from 11.5% to 50%, from 40% to 86% on the two shared tags and from 7% to 44% on the new ones. The word error rate on the tagged sentences also falls, from 8.11% to 6.73%, so the fine-tuned model reads the words more accurately while adding the sounds, and its speaker similarity stays close to the released model's (0.74 against 0.75). On plain text without tags (Table 2), the cost is small: the word error rate rises from 3.93% to 4.78%, and speaker similarity is the same, 0.84 for both models.

Table 2: Zero-shot Vietnamese TTS on plain text, without NVV tags. Every system is evaluated on the same 238 utterances sampled from our internal VietVoice dataset. WER is in %.

System	**WER ↓**	**SIM-o ↑**	**RTF ↓**
viXTTS	3.27	0.70	0.67
F5-TTS	3.02	0.82	0.47
VietVoice-TTS	2.37	0.78	0.47
VieNeu-TTS	2.00	0.65	0.89
OmniVoice			
+ base	3.93	0.84	0.06
+ fine-tuned (ours)	4.78	0.84	0.06

Table 3: Vietnamese NVV synthesis, over all 2,940 tagged items (14 classes). The two OmniVoice models get the same tagged input, so for 12 of the 14 classes the base model sees tags it was never trained on. WER is in %.

System	**NVMR ↓**	**NVPA ↑**	**WER ↓**	**SIM-o ↑**	**RTF ↓**
Fish Audio	XX	XX	XX	XX	XX
OmniVoice					
+ base	0.6435	0.1153	8.11	0.75	0.06
+ fine-tuned (ours)	0.3507	0.5000	6.73	0.74	0.06

Table 4: Listening test on NVV synthesis, on a 1–5 scale with 95% confidence intervals. MOS-Q: audio quality; MOS-N: naturalness; MOS-E: expressiveness.

System	**MOS-Q ↑**	**MOS-N ↑**	**MOS-E ↑**
Fish Audio	XX±XX	XX±XX	XX±XX
OmniVoice			
+ base	XX±XX	XX±XX	XX±XX
+ fine-tuned (ours)	XX±XX	XX±XX	XX±XX

4 Conclusion

We set out to give Vietnamese TTS the ability to laugh, sigh, cough and make other non-verbal sounds where the text asks for them. The released OmniVoice cannot do this: on our Vietnamese test sentences it made no sound at all for 64% of the tags and the right sound in the right place for only 11.5%. Because no Vietnamese training data for this task existed, we built a pipeline that creates it automatically from real Vietnamese speech, adapting SynParaSpeech with a larger sound inventory, an audio model that places each tag in a real pause, speaker-matched sound selection, natural pauses around the inserted sound and automatic quality checks. The pipeline produced about 811 hours of labelled speech covering 17 sound types. Fine-tuned on this data, OmniVoice makes the right sound at the right place for 50% of tags, misses 35% instead of 64%, and reads tagged sentences more accurately (WER 6.7% against 8.1%), at a small cost on plain text (WER 4.8% against 3.9%, with the same speaker similarity) and with a real-time factor of 0.06, fast enough for live conversation.

The work has limits that the next steps should address. Each training example carries one sound, inserted into an existing pause, so sounds that overlap speech, such as laughing while talking, are not covered. The inserted sounds are recorded by other people and only converted into the target voice, so their style may not always suit a Vietnamese speaker. Three of the 17 sound types (burp, grunt and yell) are trained but not yet tested, and several detection thresholds in the evaluation are still to be calibrated by listening. Next, we plan to complete the listening tests against Fish Audio and the released OmniVoice, to train a version on ViVoice alone to separate the effect of the NVV data from that of extra Vietnamese training, and to extend the test set to all 17 sound types.

REFERENCES

Bingsong Bai, Qihang Lu, et al. SynParaSpeech: Automated synthesis of paralinguistic datasets for speech generation and understanding. *arXiv preprint arXiv:2509.14946*, 2025.

capleaf. viVoice: Enabling Vietnamese multi-speaker speech synthesis. `https://huggingface.co/datasets/capleaf/viVoice`, 2024.

Fish Audio. OpenAudio S1. `https://huggingface.co/fishaudio/openaudio-s1-mini`, 2025.

Eduardo Fonseca, Xavier Favory, Jordi Pons, Frederic Font, and Xavier Serra. FSD50K: An open dataset of human-labeled sound events. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:829–852, 2022.

Yuan Gong, Jin Yu, and James Glass. VocalSound: A dataset for improving human vocal sounds recognition. In *Proc. ICASSP*, 2022.

Gabriel Mittag, Babak Naderi, Assmaa Chehadi, and Sebastian Möller. NISQA: A deep CNN-self-attention model for multidimensional speech quality prediction with crowdsourced datasets. In *Proc. Interspeech*, 2021.

Han Zhu, Lingxuan Ye, Wei Kang, Zengwei Yao, Liyong Guo, Fangjun Kuang, Zhifeng Han, Weiji Zhuang, Long Lin, and Daniel Povey. OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models. *arXiv preprint arXiv:2604.00688*, 2026.

A TRAINING-DATA SYNTHESIS DETAILS

A.1 NVV CLIPS

Besides VocalSound (Gong et al., 2022), the pool draws on two kinds of source: (1) other NVV corpora: Nonspeech7k, whose clips we use as they are, and FSD50K (Fonseca et al., 2022), whose clip-level labels we localise in time; and (2) NVV events cut from an existing tagged speech corpus, NonVerbalSpeech-38K (NVS-38K). From Nonspeech7k we take breathing, crying, screaming and yawn; its other classes are already covered by VocalSound. For VocalSound we keep only the test and validation splits (§3.1).

Each source locates its events differently. For NVS-38K, whose captions carry one inline tag, we force-align the caption words with Qwen3-ForcedAligner. The aligner tends to stretch the neighbouring words over the non-verbal sound, so the gap between the two words around the tag does not give the event's extent. It only confirms that the tag refers to the dataset's own detected non-verbal region (within 1 s), and that region is what we cut. FSD50K has clip-level labels only, so FlexSED's frame posterior for the class prompt locates the events (frames ≥ 0.5, gaps up to 0.3 s merged). Clips that carry two of our target labels are skipped, since their events could not be told apart.

Every located event is then cut and checked in the same way. Its edges are tightened to the frame energy within ± 0.2 s of the region, then padded by 75 ms and faded by 10 ms. The event is kept only if (i) FlexSED scores its class ≥ 0.5 over the sound and scores no VocalSound class higher, so a new class does not duplicate an existing one; (ii) no other labelled sound of the recording scores ≥ 0.5 over it; (iii) Qwen3-ASR finds no lexical word in it, so no speech overlaps it; (iv) its length is within a per-class range (e.g. 0.2–2.5 s for breathing, 0.8–4.0 s for a yawn); (v) its SNR against the recording's noise floor is at least 6 dB; and (vi) at most 0.1% of its samples are clipped. Table 5 lists the resulting 17 classes.

Table 5: NVV clip pool per tag. The first six classes come from VocalSound. Hours are the summed clip durations.

Tag	Clips	Hours
`laughter`	909	1.07
`sigh`	907	1.15
`cough`	907	1.15
`throatclearing`	907	0.97
`sneeze`	907	0.98
`sniff`	909	1.02
`breathing`	3,196	2.28
`crying`	2,288	2.18
`screaming`	896	0.71
`gasp`	364	0.06
`burp`	316	0.09
`yawn`	309	0.20
`snoring`	210	0.09
`grunt`	69	0.01
`yell`	37	0.01
`moan`	23	0.01
`pant`	11	0.00
Total	13,165	11.96

A.2 SLOT FINDING

On a 20-utterance pilot, placing the tag first and aligning it afterwards put the nearest real pause a median 2.02 s away from the aligned tag. Silero VAD finds the speech regions. Regions less than 0.10 s apart are merged, so stop closures and glottal breaks do not count as pauses, and the remaining gaps are trimmed by 0.05 s at each edge and kept if at least 0.12 s long. Silence before the first word and after the last is ignored. ViVoice transcripts have no word timing, so Qwen3-ForcedAligner aligns the untagged transcript to the audio. A pause becomes a slot when it overlaps the gap between two aligned words; if two pauses fall in one gap, the longer is kept. Each slot records its midpoint and the character offset of the word after it. If any word cannot be matched back to the transcript in order, the utterance gets no slots and is skipped, rather than risk a tag between the wrong two words.

A.3 TAG SELECTION

The recording and the marked transcript go to Qwen3-Omni-30B-A3B-Instruct, served locally with vLLM. The model is asked to listen to the delivery and answer in JSON with the one marker where a non-verbal sound would most plausibly occur and the one tag from the offered list that fits what the speaker is doing there, or `{"slot": null}` if no position suits any sound. We sample with temperature 0.2 and top-p 0.95. An invalid reply is retried, up to three attempts in all, and a null answer skips the utterance.

On the pilot the model chose only laughter and sigh. The tag balancer keeps a running count of the tags assigned so far, carried across runs, and withholds from the offered list any tag that holds more than XX% of them, with the share taken over at least XX tags. A reply that names a withheld tag is re-asked. The tag is offered again once the other tags catch up. With the balancer, the training set holds between 10,190 and 13,522 utterances per tag (Table 1).

A.4 CLIP SELECTION

Clips whose peak is below 0.05 or that are shorter than 0.35 s after trimming silence, and clips with more than 1% of samples at full scale, are excluded. The four measures are taken on each trimmed clip and on the 2.5 s of speech before the cut: *intensity*, the mean of the clip's crest-factor and duration ranks within its class, against the rank of the context window's RMS among all windows of that length in the utterance; *clarity*, the SNR as the ratio of the 90th to the 10th percentile of 20 ms frame RMS (0–60 dB, measured on the whole utterance for the speaker); *tempo*, envelope peaks per

second, compared in octaves and used for laughter only, since the timing of a cough or sneeze has nothing to do with speaking rate; and *tilt*, the slope of a line fitted to the log envelope, in dB from start to end. Each distance is divided by a tolerance set to about a third of its measured spread (0.35 rank, 12 dB, 1 octave, 18 dB). The weights are $\lambda = 1.0, 1.5, 1.0, 0.5$ for intensity, clarity, tempo and tilt. Clarity weighs most because VocalSound's microphones vary far more than its speakers' loudness. Tilt has half weight because its premise, that a speaker trailing off should be followed by a decaying event, has not been confirmed by listening. So that a soft-spoken speaker still sometimes gets a loud laugh, the intensity target is drawn per row from $\mathcal{N}(r, 0.2^2)$, clipped to $[0, 1]$, around the speaker's own rank r. Clip features are precomputed once for the whole pool, so every row samples from the full class.

A.5 VOICE CONVERSION

The selected clip is trimmed of silence, peak-normalised to 0.95 and zero-padded to at least 1 s, as Seed-VC's input chunking requires. It is converted with Seed-VC v1 (50 diffusion steps, classifier-free guidance rate 0.7, length adjustment 1.0, F0 conditioning off) at 22.05 kHz, and is trimmed again afterwards. The pitch tracker that F0 conditioning would use is RMVPE.

A.6 SPLICING

The converted event's RMS is set to the RMS of the 2.5 s of speech before the cut plus a per-class offset (laughter $+3$ dB, sneeze $+4$, cough $+2$, throat clearing -2, sigh -4, sniff -5, and 0 for the other classes), and its peak is capped at 1.5 times the speech's 99.9th-percentile level. The recording is cut at the slot's midpoint, and a 50 ms exponential fade (curvature k=5) is applied to all four edges: the end of the speech before the cut, both ends of the event and the start of the speech after it. An exponential fade reaches half level within the first 14% of its length.

In the target pauses, $\bar{T}_p$ is the median gap between voiced stretches of the utterance, clamped to $[0.04, 0.40]$ s, and s_g=1. The class scales $(s^{(c)}_{\text{pre}}, s^{(c)}_{\text{post}})$ are laughter $(0.6, 1.4)$, sigh $(0.5, 1.2)$, cough and throat clearing $(0.7, 1.0)$, sneeze $(0.8, 1.3)$, sniff $(0.5, 0.7)$, and $(1, 1)$ for the other classes. The inserted silence is $\max(0, T - T_{\text{exist}})$, where T_{exist} is the silence already on that side of the cut. It is filled with the recording's quietest 100 ms, tiled with crossfaded seams, since generated white noise audibly hissed. Finally the whole recording is peak-normalised to 0.95 and written as 16-bit PCM at 22.05 kHz.

A.7 FILTER

A row is kept if (1) its NISQA MOS drops by ≤ 0.5 from source to result, (2) its NISQA discontinuity drops by ≤ 0.22, catching an audible seam at the splice, and (3) its boundary activity at the cut is ≤ 1.2. Boundary activity is measured on the source recording as the RMS of the 50 ms on each side of the cut divided by the RMS of that whole side (0 means the cut is in silence, about 1 means it is in speech of average loudness), and a row is judged by its worse side. The thresholds were set on the 20-utterance pilot: the MOS threshold removes one clear outlier, and the other two were chosen by listening. Seed-VC's diffusion is not seeded, so a rejection means that one conversion was bad, and re-rendering a rejected row may produce a passing take.

B EVALUATION DETAILS

B.1 TEST SET

A tag fits a sentence when the sentence describes the situation that produces the sound: being startled for a gasp, falling sound asleep for snoring, or pepper or bitter cold for a sneeze. The tag sits after the action word or at a comma. Of the 420 tags, 281 (67%) are mid-sentence, 116 at the end and 23 at the start; throat clearing leans towards the start of the sentence, and snoring, sniffing and screaming towards the end. 92 sentences were lightly edited to fix spelling or cut a garbled ending. Sentences are 7–48 words long (median 14) and contain almost no English words. 21 sentences contain digits, which the reference keeps but the ASR writes out as words, so they count as errors: without them, the released model's WER falls from 8.11% to 6.94%.

B.2 METRIC PROCEDURES

NVPA and NVMR. The tag is removed and the remaining words are force-aligned to the 16 kHz output with a Vietnamese wav2vec2 CTC model (`nguyenvulebinh/wav2vec2-base-vietnamese-250h`) and torchaudio's `forced_align`. Each word boundary is a slot, and the tag sits at one of them. Qwen3-ForcedAligner is not used here: besides stretching words over non-verbal sounds (Appendix A), on Vietnamese it collapses runs of words to one instant. FlexSED then scores each class prompt over the whole recording, in 10 s windows with a 5 s hop, and each frame keeps its highest posterior across windows. A class's events are runs of frames above 0.2 that reach the class threshold at least once. The threshold is 0.5 for most classes, 0.4 for throat clearing, and 0.7 for breathing, which FlexSED also scores at 0.5–0.7 on pauses where no breath is heard. Runs less than 0.12 s apart are merged, events shorter than 0.08 s are dropped, and events shorter than 0.25 s that touch either end of the clip are ignored as cut-off tails. A tag is *correct* if an event of its class lies within 3 s of its slot with no aligned word in between; a word that overlaps the event by at most 0.05 s does not count as in between. Otherwise the tag is *misplaced* if exactly one word separates the nearest event of its class from the slot, *wrong class* if only another class fires at the slot, and *missed* if no event of any class is at the slot. Classes are scored strictly: a sigh, pant or gasp also triggers FlexSED's breathing prompt, but counts as correct only when its own class fires.

WER. The ASR model is `sherpa-onnx-zipformer-vi-2025-04-20` with greedy search on the 16 kHz output. The reference and the transcript are normalised alike: tags are removed, the text is put in Unicode NFC and lower-cased, and every character other than a letter or digit becomes a space. WER is computed at corpus level over all items of a system, not as a mean of per-item WERs.

SIM-o. The speaker model is `speechbrain/spkrec-ecapa-voxceleb`. Both recordings are embedded at 16 kHz and the embeddings are L2-normalised (0–1 in practice).